



UNIVERSITÀ DI PISA

University of Pisa

Competitive Programming and Contests

Academic Year 2025–2026

HANDS-ON REPORT

Solution to Hands-On 3

Problem #1: Holiday Planning

Problem #2: Design a course

This report presents the solutions for the third hands-on assignment of the course. Both problems are solved with dynamic programming. The first problem is a two-dimensional knapsack problem, while the second problem is a two-dimensional longest increasing subsequence problem.

Name	Oleksiy Nedobiychuk
Matricola	597455
Course	Competitive Programming and Contests
Date	July 13, 2026

Introduction

Here, I present the solutions to the two problems of Hands-On 3. Both problems ask for an optimal selection under constraints that grow with the input, and in both cases a greedy choice is not enough to guarantee optimality, so I solve them with dynamic programming.

Exercise 1: Holiday Planning

The solution is implemented in `hands_on31.rs`. The input gives n cities and d available days, followed by one line per city with the number of attractions that can be visited on each of the d days of that city's itinerary. Attractions on a given day of a city can only be visited if all the earlier days of that same city are also visited, so for a city the useful quantities are prefix sums of the daily attraction counts rather than the raw counts.

The array `sums` stores exactly these prefix sums: `sums[c][k]` is the number of attractions collected by spending k days in city c . It is built once, in $O(n \cdot d)$ time, before the main recurrence.

The main table `mrs` has size $(n + 1) \times (d + 1)$, and `mrs[c][j]` is the best number of attractions obtainable using only the first c cities and a total budget of j days. The first row and the first column are zero, since zero cities or zero days give zero attractions. For $c \geq 1$ and $j \geq 1$, the recurrence tries every possible number of days k spent in city c , with the remaining $j - k$ days assigned to the first $c - 1$ cities:

$$\text{mrs}[c][j] = \max_{0 \leq k \leq j} (\text{mrs}[c-1][j-k] + \text{sums}[c][k]).$$

Because `mrs[c][j]` only depends on row $c - 1$ and on smaller values of j in row c , the loops are ordered with the city index outermost, the day budget j in the middle, and the split k innermost. The final answer is read directly from `mrs[n][d]`.

Filling the `sums` table costs $O(n \cdot d)$. The main table has $O(n \cdot d)$ cells, and each cell is computed in $O(d)$ time because of the innermost loop over k , giving a total running time of $O(n \cdot d^2)$ and $O(n \cdot d)$ memory.

Exercise 2: Design a Course

The solution is implemented in `hands_on32.rs`. Each topic has a beauty value and a difficulty value, and the goal is to select the longest sequence of topics for which both values increase together. Since the professor is free to order the lectures of the course however he likes, the input order of the topics does not matter, and the problem reduces to a two-dimensional version of the longest increasing subsequence.

The topics are first sorted by beauty in ascending order, breaking ties by difficulty in descending order (Reverse on the second key). Sorting difficulty in the opposite direction on ties prevents two topics with equal beauty from both being extended into the same increasing run, since such a pair could never both belong to a valid course anyway. After this sort, the problem is exactly the one-dimensional longest increasing subsequence on the difficulty values, read in the new sorted order.

The subsequence is computed with the usual patience-sorting technique. The array `d` holds, for each achieved length l , the smallest possible difficulty value that ends a selected sequence of that length; `d[0]` is initialized to `i32::MIN` as a sentinel. For every topic, in sorted order, `partition_point` performs a binary search on `d` to find the position l where the current difficulty value would extend the run. The value is written into `d[l]` only when it is strictly greater than `d[l-1]` and strictly smaller than the value currently stored at `d[l]`, and `res` is updated with the largest length reached so far.

Sorting the topics costs $O(n \log n)$. The main loop performs one binary search per topic, so it also costs $O(n \log n)$. The overall running time is $O(n \log n)$, with $O(n)$ additional memory for the d array.

Testing

The implementations were validated with the provided input and output files. The first binary was checked against the files in `data/test_handson31`. The second binary was checked against the files in `data/test_handson32`. In both cases, the produced output was compared with the expected output using `diff`.